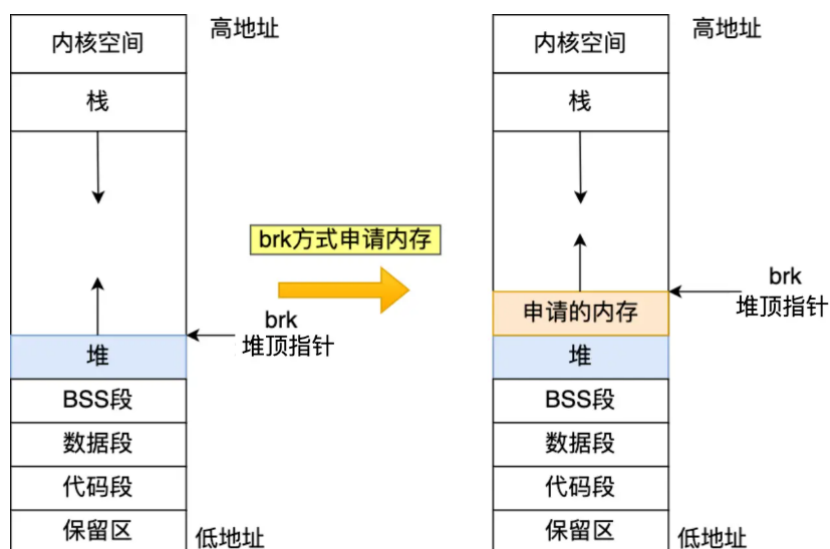


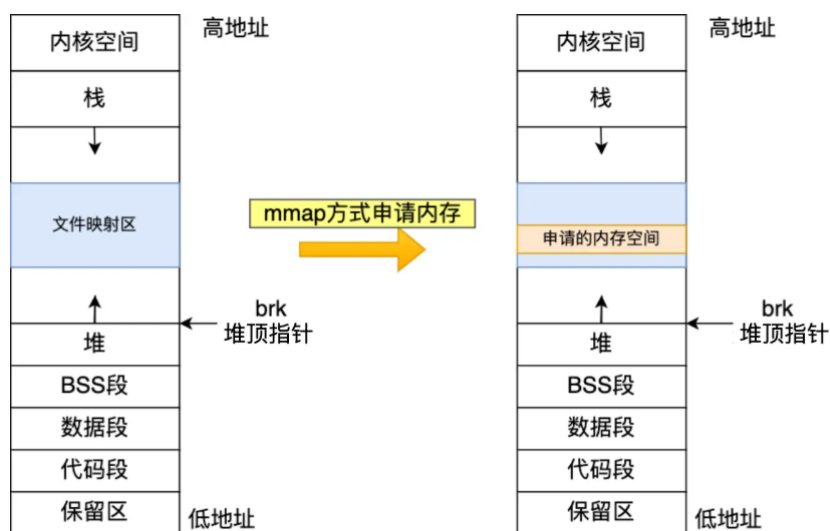
7.28小组模拟面

1.mmap底层

mmap是c库函数，里面主要是申请空间，如果申请空间小于128K，用brk申请，具体操作如下



对于大于128K的采用mmap，从库中申请一片空间



malloc分配的不是物理内存，而是虚拟内存，只有当分配的虚拟内存被访问，虚拟内存才会映射到物理内存（查页表，发现对应页不在物理内存，触发缺页中断，建立映射关系），占用物理内存空间

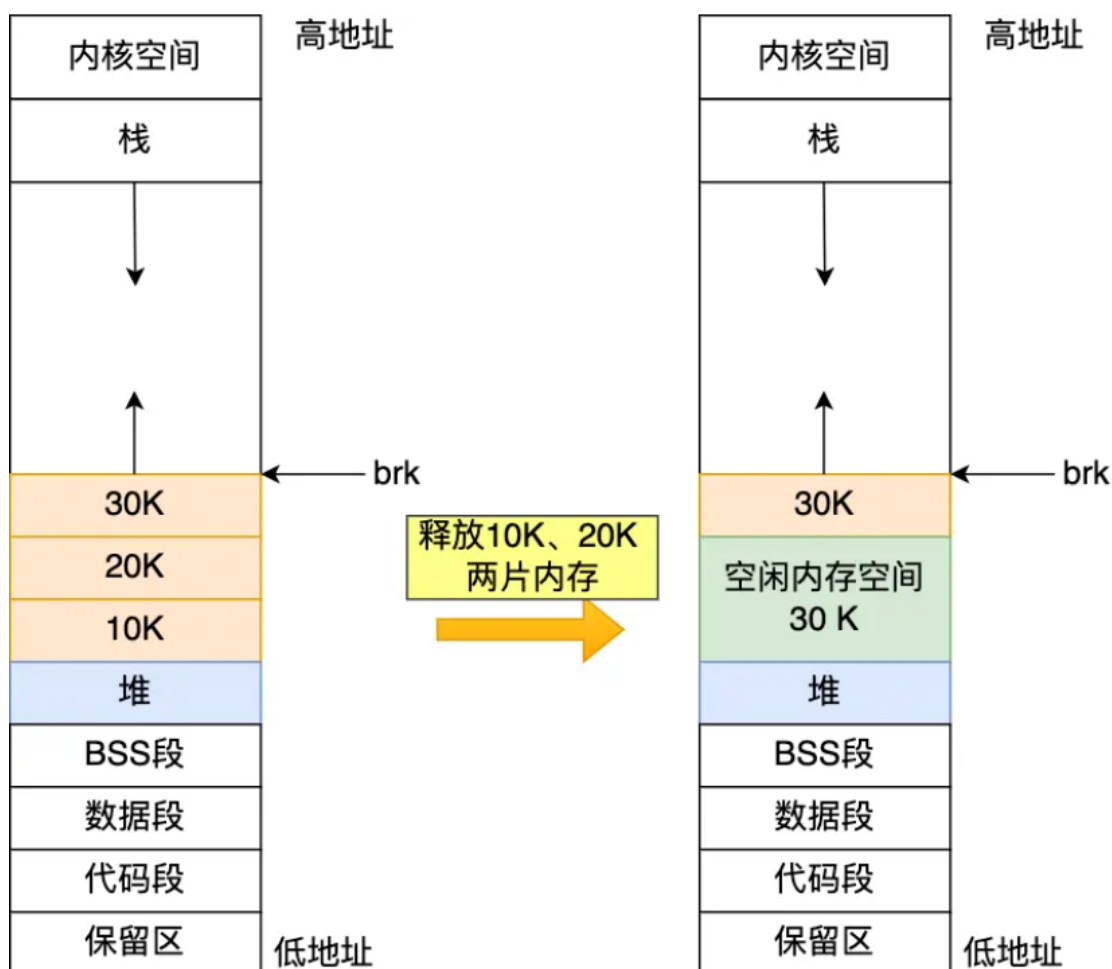
ptmalloc

由于brk申请的空间都小于128k，属于小碎片，且申请的空间在堆区，所以第一次进行malloc时，会一次性申请大量空间，同时维护一个**ptmalloc内存池**，**这样后续所有申请和释放空间都和内存池交互，可以减少系统调用，其内部是一个碎片链表**，会定期对小碎片进行合并，需要申请内存时，遍历链表查找合适的碎片空间并返回，释放内存时，**如果用户释放的小于128k，就进入内存池，否则申请的空间来自于库（mmap申请），直接归还操作系统**

为什么不全用mmap申请空间？

brk底层维护ptmalloc线程池，减少了和操作系统的交互，且里面的内存块还可能存在于物理内存中（使用过），这样的话就不会发生缺页中断。反之mmap申请的空间一定会先从用户态到内核态，然后申请的空间第一次调用一定是缺页中断的

为什么不全用brk申请空间？



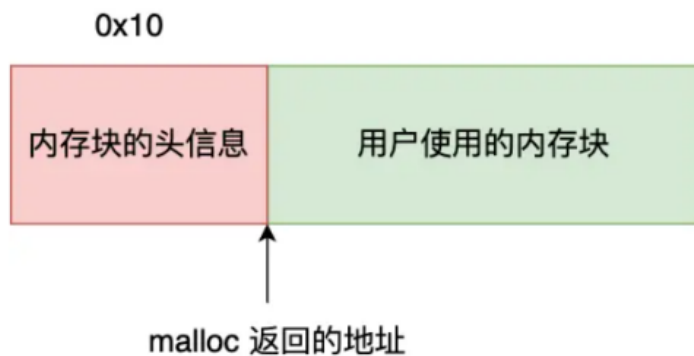
因为可能产生细小碎片，如图，大于30K的空间申请时，brk指针还会向上偏移

为什么free函数可以精准释放内存（只给内存地址），而不需要传入需要释放的内存大小

free() 函数只传入一个内存地址，为什么能知道要释放多大的内存？

还记得，我前面提到， malloc 返回给用户态的内存起始地址比进程的堆空间起始地址多了 16 字节吗？

这个多出来的 16 字节就是保存了该内存块的描述信息，比如有该内存块的大小。



这样当执行 free() 函数时，free 会对传入进来的内存地址向左偏移 16 字节，然后从这个 16 字节的分析出当前的内存块的大小，自然就知道要释放多大的内存了。

2.类与类之间的关系？你了解的设计原则？

类与类之间的关系？

- 继承 / 实现（接口）
- 关联（一个类做另一个类的成员变量）
- 依赖（一个类做另一个类的**方法参数**）
- 组合（同关联关系，整体与部分关系，两个类生命周期相同，比如人和四肢）
- 聚合（同关联关系，但是两个类的生命周期不同，比如电脑和cpu）

设计原则（7个）：

1.单一职责	功能单一，方便组合和复用
2.开闭原则	对扩展开放，对修改关闭。
3.里氏代换原则	任何基类可能出现的地方，子类一定可以出现
4.依赖倒转原则	针对接口编程，依赖于抽象而不依赖具体
5.接口隔离原则	使用多个隔离的接口，比使用单个接口更好
6.迪米特法则（最少知道原则）	
一个实体应当尽量少的和其他实体之间发生相互作用，使得系统功能模块相对独立	
7.合成复用原则	尽量使用合成/聚合的方式，而不是使用继承

- **单一职责**：一个类或函数只实现一个功能，且尽量减少调用别的类中方法或函数，**降低耦合度**
- **开闭原则**：对新的需求，增加新的接口并实现，而不是改动原来的接口
- **里氏代换原则**：子类可以替代父类
- **接口隔离原则**：一个接口实现的内容和另一个接口实现的内容要没有交集（有点像单一职责）
- **合成复用原则**：尽量用组合而不是继承，比如我要用一个类中的方法，我就最好别创建一个新的子类继承他，而是创建一个他的对象，然后直接用他的方法
- **迪米特法则**：类与类之间避免过多耦合，一切信息传递通过中介者
- **依赖倒转原则**：如果需要在在一个类中引入另一个类，则**引入该类的抽象，后续修改只用修改抽象类**：比如定义一个用户类，用户具备说话的方法，最开始是说英语，后来有了说其他语言的需求，如果要修改，则该方法需要大改，但假如我把这个方法抽象为一个类，则不用修改这一部分，只需要修改这个抽象类

3.常考设计模式，项目中用到的设计模式？

什么是设计模式：针对软件开发中反复出现的问题总结出的一种通用解决方案模版

1.单例模式：

单例模式实现的几个要素：

- 删除拷贝构造和赋值构造
- 构造函数设置为私有
- 提供公有接口函数访问唯一单例

懒汉式（存在多线程安全问题）：等到需要创建相应类实例时再创建

```

1  #include <iostream>
2  #include <mutex>
3
4  class Singleton {
5  public:
6      static Singleton& getInstance() {
7          if (instance == nullptr) { // 非线程安全！多线程可能创建多
            个实例
8              instance = new Singleton();
9          }
10         return *instance;

```

```

11     }
12
13     Singleton(const Singleton&) = delete;
14     Singleton& operator=(const Singleton&) = delete;
15
16 private:
17     Singleton() { std::cout << "Singleton (懒汉式) 构造完成\n";
18     }
19
20     static Singleton* instance; // 未初始化的指针
21 };
22
23 // 初始化静态指针
24 Singleton* Singleton::instance = nullptr;

```

饿汉式：单例类开始时就实例化出对象，后续需要时再通过接口调用，**由于发生在编译期，所以没有线程安全问题**

```

1  #include <iostream>
2  #include <mutex>
3
4  class Singleton {
5  public:
6      // 获取单例实例（直接返回静态变量）
7      static Singleton& getInstance() {
8          return instance; // 静态变量在程序启动时已构造
9      }
10
11     // 删除拷贝构造和赋值操作（防止复制单例）
12     Singleton(const Singleton&) = delete;
13     Singleton& operator=(const Singleton&) = delete;
14
15 private:
16     // 私有构造函数（防止外部创建实例）
17     Singleton() { std::cout << "Singleton (饿汉式) 构造完成\n";
18     }
19
20     // 静态实例（程序启动时直接构造）
21     static Singleton instance;
22 };
23
24 // 在类外定义静态成员（程序启动时初始化）
25 Singleton Singleton::instance;

```

懒汉式和饿汉式使用场景PK:

- 饿汉式:

优点: 保证线程安全

缺点: 如果单例实例占用资源较多, 开始初始化后长期不使用, 则造成资源浪费, 比如创建一个单例实例用于连接数据库, 后续很长一段时间都不进行数据库操作, 就占用了数据库资源。 并且影响程序启动速度

- 懒汉式

优点: 延迟加载, 避免资源占用, 灵活性高

缺点: 多线程并发问题, 需要用双检索或call_once规避

懒汉式--双检锁版本

```
1  #include <iostream>
2  #include <mutex>
3
4  class Singleton {
5  public:
6      static Singleton& getInstance() {
7          Singleton* tmp = instance; // 第一次检查（无锁，非原子操作）
8
9          if (tmp == nullptr) {
10             std::lock_guard<std::mutex> lock(mtx); // 加锁
11             tmp = instance; // 第二次检查（加锁后，但无内存序保护）
12             if (tmp == nullptr) {
13                 tmp = new Singleton(); // 构造实例
14                 instance = tmp; // 存储结果（非原子写入）
15             }
16             return *tmp;
17         }
18
19         Singleton(const Singleton&) = delete;
20         Singleton& operator=(const Singleton&) = delete;
21
22     private:
23         Singleton() { std::cout << "Singleton（有问题的双检锁）构造完成\n"; }
24
25         static Singleton* instance; // 普通指针（非原子）
26         static std::mutex mtx; // 互斥锁
```

```

27     };
28
29     // 初始化静态成员
30     Singleton* Singleton::instance = nullptr; // 普通指针初始化
31     std::mutex Singleton::mtx;

```

所谓双检锁，就是判断两次是否需要加锁，第一次检查是为了快速判断指针是否为空（不加速），为空则具体进行加锁，然后再判断指针是否为空。**由于上锁行为同一时间只有一个线程能操作，所以如果有多个线程同时上锁，只有一个线程能具体构造对象，其余线程获取到锁时对象已经被初始化，此时无法通过第二次锁的检验**

普通双检锁解决了多线程并发下对象创建的问题，**但仍存在访问对象问题（指令重排）**：因为new命令在编译器角度，分为三步

- 1.分配内存（malloc）
- 2.将指针指向新内存
- 3.调用构造相应类的函数

根据编译器不同，三条指令顺序可能不同

- 如果是1->3->2，则没有问题，因为当一个线程申请到锁资源后，哪怕时间片耗尽，**都是最后一步才将指针指向新内存**，所以在此期间，其他线程都没有锁，无法创建单例对象，此时对象状态也显示为空（指针），无法正常使用，等到创建对象的线程重新获取时间片，创建好对象后其余线程才能访问
- 如果是1->2->3，则可能出现创建对象的线程时间片耗尽，刚好做完第二步，此时指针已经指向空间，但类的构造函数还没调用，其余线程一看对象有了，**直接使用，却由于没初始化，于是产生了未定义行为的错误**

```

1  #include <iostream>
2  #include <mutex>
3  #include <atomic>
4
5  class Singleton {
6  public:
7      static Singleton& getInstance() {
8          Singleton* tmp =
instance.load(std::memory_order_acquire); // 第一次检查（无锁）
9          if (tmp == nullptr) {
10              std::lock_guard<std::mutex> lock(mtx); // 加锁
11              tmp = instance.load(std::memory_order_relaxed);
// 第二次检查（加锁后）

```

```

12         if (tmp == nullptr) {
13             tmp = new Singleton(); // 构造实例
14             instance.store(tmp,
std::memory_order_release); // 存储结果
15         }
16     }
17     return *tmp;
18 }
19
20 Singleton(const Singleton&) = delete;
21 Singleton& operator=(const Singleton&) = delete;
22
23 private:
24     Singleton() { std::cout << "Singleton (双检锁) 构造完成\n";
25 }
26
27     static std::atomic<Singleton*> instance; // 原子指针
28     static std::mutex mtx; // 互斥锁
29 };
30 // 初始化静态成员
31 std::atomic<Singleton*> Singleton::instance{nullptr};
32 std::mutex Singleton::mtx;

```

如何真正解决指令重排问题：将操作变成原子操作，也就是new内部的三个指令，确保一次性完成（具体做法：把构造过程封装成函数，**通过call_once调用函数**（call_once保证函数代码只执行一次，内部实现了多线程安全））

```

1  #include <iostream>
2  #include <mutex>
3
4  class Singleton {
5  public:
6      static Singleton& getInstance() {
7          std::call_once(flag, []() { // 保证初始化代码仅执行一次
8              instance = new Singleton();
9          });
10         return *instance;
11     }
12
13     Singleton(const Singleton&) = delete;
14     Singleton& operator=(const Singleton&) = delete;
15

```



```

16 private:
17     Singleton() { std::cout << "Singleton (call_once) 构造完成\n"; }
18
19     static Singleton* instance; // 静态指针
20     static std::once_flag flag; // 一次性标志
21 };
22
23 // 初始化静态成员
24 Singleton* Singleton::instance = nullptr;
25 std::once_flag Singleton::flag;

```

2.观察者模式：

角色	职责
Subject (被观察者)	维护观察者列表，提供注册/注销接口，状态变化时通知所有观察者。
Observer (观察者)	定义更新接口（如 update()），接收被观察者的通知并执行相应操作。
ConcreteSubject (具体被观察者)	实现 Subject 接口，存储具体状态，状态变化时主动调用 notifyObservers()。
ConcreteObserver (具体观察者)	实现 Observer 接口，定义收到通知后的具体行为（如更新数据、刷新界面）。

具体可以看作这样的交互模式（真实情况是1对多，不是1对1）：真实交互的是观察者中的具体观察者和被观察者中的被观察者

观察者模式和中介者模式的区别：中介者模式是中间件同时和两端交互，观察者模式中观察者和被观察者都有一个具体观察的对象交互，得到消息后再给自己的对应对象



是一种行为模式：用于**建立对象间一对多的依赖关系**，当一个对象（被观察者）状态发生变化时，所有依赖它的对象（观察者）会自动收到通知并更新

使用例子：

- 比如qt的信号槽机制：信号发出后，元对象编译器（MOC）就会通过映射表找到对应槽函数的索引，然后由索引转到moc文件中的函数，通过事件分发机制转交给对应槽函数所在线程
- redis中的订阅发布机制
- 消息队列（类似第二点），在先更新数据库再删除缓存操作中，删除缓存的操作必须成功（否则可能出现脏数据），所以此时通过订阅MySQL中的binlog（具体是Canal中的MQ中间件把自己伪装成MySQL从节点，获取主节点的binlog日志中有关缓存的部分，通知主节点删除缓存），直到获得主节点删除成功的ack，这时候再把日志中的记录删除

观察者模式用于解决什么问题？引入后又可能带来什么新问题？

解决问题：通过搭建中间件，实现一对多通信，避免了程序高耦合，符合开闭原则，同时一个事件可以触发多个独立响应，避免了重复代码

新问题：观察者获取事件后处理没有顺序，如果修改全局变量可能造成并发问题，观察者过多，可能性能开销比较大

模拟实现：

```

1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  // 前向声明
6  class Observer;
7
8  // 被观察者抽象接口
9  class Subject {
10 public:
11     virtual void attach(Observer* observer) = 0;    // 注册观察
12     virtual void detach(Observer* observer) = 0;    // 注销观察
13     virtual void notifyObservers() = 0;             // 通知所有
14     virtual ~Subject() = default;
15 };
16
17 // 观察者抽象接口
18 class Observer {

```

```

19 public:
20     virtual void update(int value) = 0;           // 接收通知
    并更新
21     virtual ~Observer() = default;
22 };
23
24 // 具体被观察者: 温度传感器
25 class TemperatureSensor : public Subject {
26 public:
27     void attach(Observer* observer) override {
28         observers_.push_back(observer);
29     }
30
31     void detach(Observer* observer) override {
32         observers_.erase(std::remove(observers_.begin(),
    observers_.end(), observer), observers_.end());
33     }
34
35     void notifyObservers() override {
36         for (Observer* observer : observers_) {
37             observer->update(temperature_); // 通知所有观察者当
    前温度
38         }
39     }
40
41     void setTemperature(int temperature) {
42         temperature_ = temperature;
43         notifyObservers(); // 温度变化时主动通知观察者
44     }
45
46 private:
47     int temperature_ = 0;
48     std::vector<Observer*> observers_;
49 };
50
51 // 具体观察者: 温度显示器A
52 class TemperatureDisplayA : public Observer {
53 public:
54     void update(int value) override {
55         std::cout << "DisplayA: Current temperature is " <<
    value << "°C" << std::endl;
56     }
57 };
58
59 // 具体观察者: 温度显示器B

```

```

60 class TemperatureDisplayB : public Observer {
61 public:
62     void update(int value) override {
63         std::cout << "DisplayB: Temperature updated to " <<
value << "°C" << std::endl;
64     }
65 };
66
67 int main() {
68     // 创建被观察者（温度传感器）
69     TemperatureSensor sensor;
70
71     // 创建观察者（显示器）
72     TemperatureDisplayA displayA;
73     TemperatureDisplayB displayB;
74
75     // 注册观察者
76     sensor.attach(&displayA);
77     sensor.attach(&displayB);
78
79     // 模拟温度变化
80     sensor.setTemperature(25); // 两个显示器都会收到通知
81     sensor.setTemperature(30);
82
83     // 注销一个观察者
84     sensor.detach(&displayA);
85     sensor.setTemperature(35); // 只有 displayB 收到通知
86
87     return 0;
88 }

```

3.中介者模式

你是怎么理解中介者模式的（举个例子）：

多个组件（部分）只与中介者交互，而不相互交互，降低了模块之间的依赖

飞机在飞行时，如果要避免碰机，必须实时和其他所有飞机通信，这样很不方便，但如果每个飞机只用和塔台通信，塔台告诉他们怎么飞，那就方便多了

为什么采用中介者模式开发项目，不采用观察者模式

我们前面提到，中介模式也是为了解耦对象之间的交互，所有的参与者都只与中介进行交互。而观察者模式中的消息队列，就有点类似中介模式中的“中介”，观察者模式的中观察者和被观察者，就有点类似中介模式中的“参与者”。那问题来了：中介模式和观察者模式的区别在哪里呢？什么时候选择使用中介模式？什么时候选择使用观察者模式呢？

在观察者模式中，尽管一个参与者既可以是观察者，同时也可以是被观察者，但是，大部分情况下，交互关系往往都是单向的，一个参与者要么是观察者，要么是被观察者，不会兼具两种身份。也就是说，在观察者模式的应用场景中，参与者之间的交互关系比较有条理。

而中介模式正好相反。只有当参与者之间的交互关系错综复杂，维护成本很高的时候，我们才考虑使用中介模式。毕竟，中介模式的应用会带来一定的副作用，前面也讲到，它有可能会产生大而复杂的上帝类。除此之外，如果一个参与者状态的改变，其他参与者执行的操作有一定先后顺序的要求，这个时候，中介模式就可以利用中介类，通过先后调用不同参与者的方法，来实现顺序的控制，而观察者模式是无法实现这样的顺序要求的。

虽然这两个模式有点像，但观察者模式和我的项目不契合，观察者模式要求和被观察者模式区分开来，也就是有明显的身份，首先我的服务器架构，我就是用的服务器做中介者，假如采用观察者模式，首先被观察的对象一定是客户，因为是客户端发生行为被服务器接收到，服务器才会去做处理，此时被观察者是多，观察者是一，和我的架构不相符

4.工厂模式

- 简单工厂

核心思想：提供一个工厂类，内部一个创建产品的函数，根据参数不同创建不同对象

- 工厂方法

核心思想：提供一个工厂接口，创建若干个工厂继承这个接口，一个工厂对应一个产品（职责单一了）

- 抽象工厂

核心思想：不再提供工厂接口，而是提供创建产品的产品接口，实现抽象工厂时只用实现其具体的产品方法，就能创建出很多不同的工厂

举个例子，假如汽车由轮胎，车壳，方向盘组成，那么我的抽象工厂就提供创建这三部分的一个接口，具体继承抽象工厂实现具体工厂时，只用实现这三部分，我就可以得到一个（红色/绿色）轮胎（红色/绿色）车壳（红色/绿色）方向盘的车（任意搭配）

在项目中如何用的设计模式

服务器那边实现了服务器类的单例模式，同时服务器板块分为三部分：中介者，网络收发部分和核心类具体处理，其中网络收发部分得到包后都会先交给中介者判断具体给核心处理类的哪部分去处理，如果是不合理的包就直接丢掉

